Minor Assignment 05 — Solutions

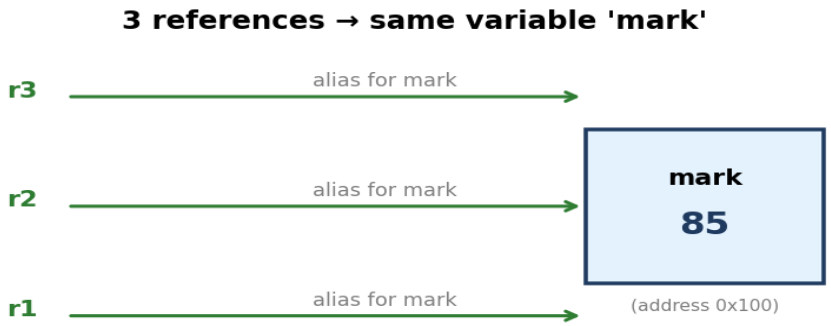
Game Programming with C++ (CSE 3545) • SFML VertexArray & C++ References • Study Guide

This guide explains every question with the concept first, then the answer. Topics covered: **C++ references** (aliases to existing variables — they let you modify the original through another name), and SFML's **VertexArray** with **Quads** primitive (the fast way to draw thousands of tiles using one texture, used to build the Zombie Arena background).

Two big ideas behind the whole assignment:
1. Reference (`int& r = x;`) — `r` *is* `x`. No new memory; changing `r` changes `x`. Must be initialised at declaration and can never be re-bound.
2. VertexArray of Quads — instead of one `Sprite` per tile (slow), we pack *all* tile corners into one array. Each quad = 4 vertices = 1 cell of the background. Each vertex carries a **position** (where to draw on screen) and a **texCoords** (which pixel of the sprite sheet to sample). One draw call = whole background.

Q1. Three references to int variable `mark`

A reference is just another name (alias) for an existing variable. You can make as many references to the same variable as you like — they all read and write the *same* memory location.



Code:

```
int mark = 85;        // the original variable

int& r1 = mark;       // r1 is an alias for mark
int& r2 = mark;       // r2 is also an alias for mark
int& r3 = mark;       // r3 is also an alias for mark

// Proof — all four names refer to the same memory:
r2 = 90;
cout << mark << " " << r1 << " " << r2 << " " << r3;
// Output: 90 90 90 90
```

Key rule: a reference must be initialised when declared (`int& r;` alone is a compile error) and cannot be made to refer to a different variable later.

Q2. Output trace — chained references

```
int main(){
    int num = 10;
    int& rnum = num;       // rnum aliases num
    int& r1num = rnum;     // r1num aliases rnum  →  also aliases num
    rnum = 100;            // writes 100 into num (and therefore all aliases)
    cout << rnum << " " << num << " " << r1num << endl;
    return 0;
}
```

Step-by-step:

- `num = 10` → memory holds 10.
- `rnum` binds to `num`.
- `r1num` binds to `rnum`, which is itself `num` — so `r1num` is *also* num.
- `rnum = 100` overwrites the single underlying memory cell. All three names now read 100.

Output:

```
100 100 100
```

Q3. Output trace — reference vs value vs pointer

```
void update(int& rnum, int vnum, int *pnum){
    rnum  = rnum  + 500;   // modifies caller's variable (reference)
    vnum  = vnum  + 500;   // modifies local copy only (pass-by-value)
    *pnum = *pnum + 500;   // modifies caller's variable (pointer)
}
int main(){
    int num1 = 11, num2 = 22, num3 = 33;
    update(num1, num2, &num3);
    cout << num1 << " " << num2 << " " << num3 << endl;
}
```

Parameter table:

Argument	How passed	Effect on caller
num1 (11)	by reference (int&)	modified → 11 + 500 = 511
num2 (22)	by value (int)	untouched (only local copy changes)
num3 (33)	by pointer (&num3)	modified through pointer → 33 + 500 = 533

Output:

```
511 22 533
```

Q4. `getMax` returning a reference

The function returns a **reference** to whichever input is larger. That means `maxVal` in `main` is an alias for the bigger of `x` and `y`. Assigning to `maxVal` rewrites that original variable.

```
int& getMax(int& a, int& b){
    return (a > b) ? a : b;       // returns a reference to a or b
}
int main(){
    int x = ?, y = ?;
    int& maxVal = getMax(x, y);   // maxVal aliases whichever is larger
    cout << maxVal << endl;
    maxVal = 30;                  // writes 30 into that larger variable
    cout << "x = " << x << ", y= " << y;
}
```

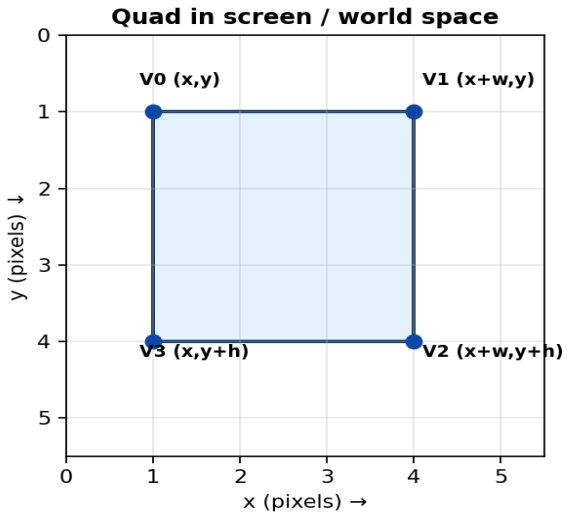
Checking the listed options:

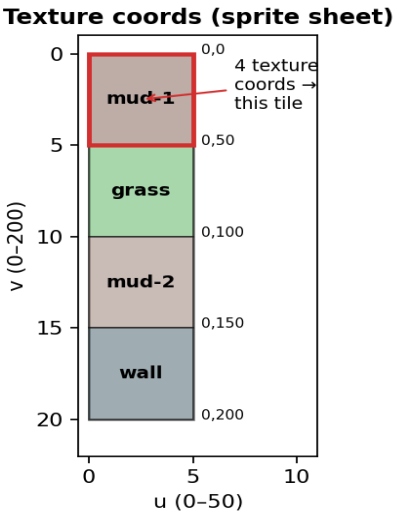
(x, y)	max printed	After maxVal=30	Final output	Listed?
10, 10	10	y=10>x, so y becomes 30	x = 10, y = 30	✗ not in list
20, 20	20	y becomes 30 (same tie rule)	x = 20, y = 30	✗ not in list
10, 20	20	y is bigger → y becomes 30	x = 10, y = 30	✗ not in list
20, 10	20	x is bigger → x becomes 30	x = 30, y = 10	✗ not in list
60, 40	60	x is bigger → x becomes 30	x = 30, y = 40	✗ not in list
40, 60	60	y is bigger → y becomes 30	x = 40, y = 30	✗ not in list

Most likely intended answer: The standard textbook example uses **x = 10, y = 20** → prints **20**, then `maxVal = 30` changes y to 30 → final line **x = 10, y = 30**. Tick the option matching whatever (x, y) your instructor assigns; the reasoning above applies to all rows.

Q5. Declare a VertexArray of Quads, size 10 × 10 × 4

We want a background made of **10 rows × 10 columns = 100 tiles**. Each tile is one **Quad**, and each Quad has **4 vertices**. So total vertices = 10 × 10 × 4 = **400**.





Code:

```
#include <SFML/Graphics.hpp>
using namespace sf;

// Declare a vertex array
VertexArray rVA;

// Set primitive type to Quads (every 4 vertices form one quad)
rVA.setPrimitiveType(Quads);

// Allocate 10 x 10 x 4 = 400 vertices
rVA.resize(10 * 10 * 4);
```

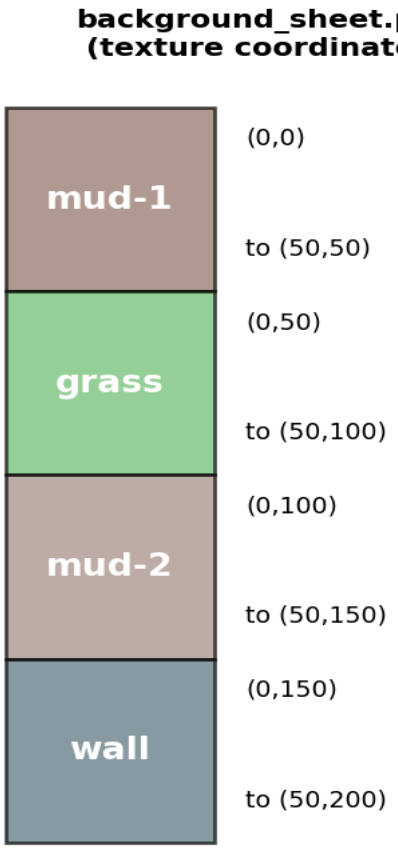
Why Quads? Each tile of the arena is a rectangle. SFML reads the vertex array in groups of 4 and connects them as filled quadrilaterals — perfect for tiled backgrounds.

Q6. Draw 3 tiles (mud-1, grass, mud-2) using the sprite sheet

Each tile occupies one Quad = 4 vertices. Every vertex needs two things:

- **position** — where on the screen the corner goes.
- **texCoords** — which pixel of the sprite sheet that corner samples.

We place the three tiles side by side on screen (each 50×50 px), and grab the matching strip of the sprite sheet for each.



Code:

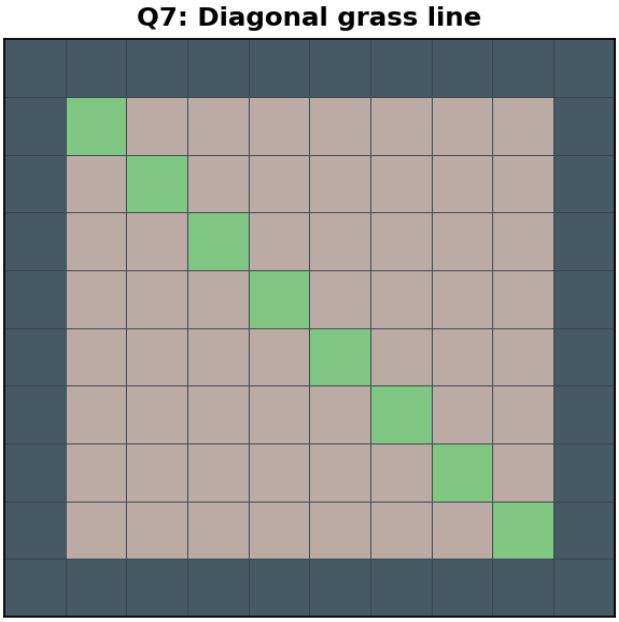
```
#include <SFML/Graphics.hpp>
using namespace sf;

// --- Setup ---
VertexArray rVA(Quads, 3 * 4);   // 3 tiles x 4 vertices = 12

Texture textureBackground;
textureBackground.loadFromFile("graphics/background_sheet.png");

const int TILE = 50;

// ============ Tile 1: mud-1 (texCoords 0,0 to 50,50) ============
// Position on screen: (0,0) to (50,50)
rVA[0].position = Vector2f(0,    0);
rVA[1].position = Vector2f(TILE, 0);
rVA[2].position = Vector2f(TILE, TILE);
rVA[3].position = Vector2f(0,    TILE);

rVA[0].texCoords = Vector2f(0,    0);
rVA[1].texCoords = Vector2f(TILE, 0);
rVA[2].texCoords = Vector2f(TILE, TILE);
rVA[3].texCoords = Vector2f(0,    TILE);

// ============ Tile 2: grass (texCoords 0,50 to 50,100) ============
```

```
// Position on screen: (50,0) to (100,50)
rVA[4].position = Vector2f(TILE,     0);
rVA[5].position = Vector2f(TILE * 2, 0);
rVA[6].position = Vector2f(TILE * 2, TILE);
rVA[7].position = Vector2f(TILE,     TILE);

rVA[4].texCoords = Vector2f(0,    TILE);       // top-left of grass strip
rVA[5].texCoords = Vector2f(TILE, TILE);
rVA[6].texCoords = Vector2f(TILE, TILE * 2);   // bottom-right
rVA[7].texCoords = Vector2f(0,    TILE * 2);

// ============ Tile 3: mud-2 (texCoords 0,100 to 50,150) ============
// Position on screen: (100,0) to (150,50)
rVA[8].position  = Vector2f(TILE * 2, 0);
rVA[9].position  = Vector2f(TILE * 3, 0);
rVA[10].position = Vector2f(TILE * 3, TILE);
rVA[11].position = Vector2f(TILE * 2, TILE);

rVA[8].texCoords  = Vector2f(0,    TILE * 2);   // top-left of mud-2 strip
rVA[9].texCoords  = Vector2f(TILE, TILE * 2);
rVA[10].texCoords = Vector2f(TILE, TILE * 3);
rVA[11].texCoords = Vector2f(0,    TILE * 3);

// --- Draw ---
window.clear();
window.draw(rVA, &textureBackground);   // one draw call, three tiles
window.display();
```

Vertex order matters. The 4 vertices of a Quad must go *around* the rectangle (top-left → top-right → bottom-right → bottom-left). If you criss-cross the order you'll get a bow-tie shape instead of a tile.

Q7. `displayBackground` — wall border + diagonal grass

The arena has: **wall** tiles on every edge, **grass** tiles along the main diagonal (i == j), and **mud-1** filling the rest. We walk every cell, decide which texture strip to use, then write 4 vertices (position + texCoords) per cell.



Code:

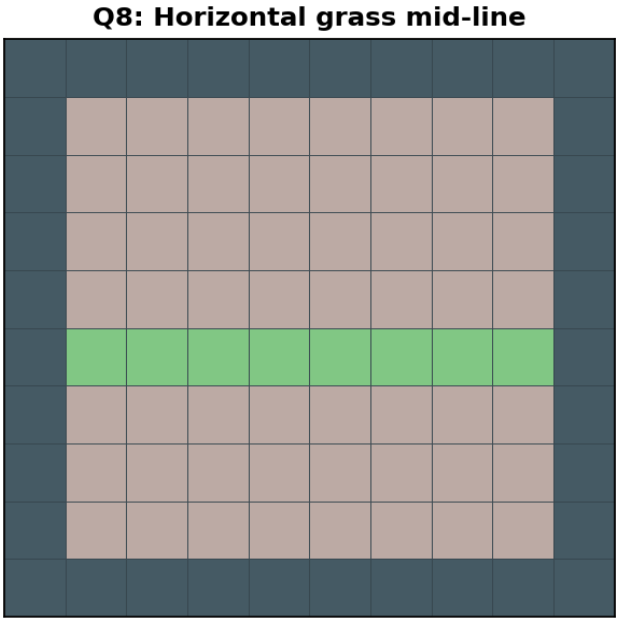
```
#include <SFML/Graphics.hpp>
using namespace sf;

int displayBackground(VertexArray& rVA, IntRect arena){
    const int TILE = 50;

    // arena.width and arena.height = pixel size of the arena.
    // Number of tiles that fit:
    int worldWidth  = arena.width  / TILE;
    int worldHeight = arena.height / TILE;

    // Resize the vertex array (4 vertices per quad)
    rVA.setPrimitiveType(Quads);
    rVA.resize(worldWidth * worldHeight * 4);

    int currentVertex = 0;
    for (int w = 0; w < worldWidth;  w++) {
        for (int h = 0; h < worldHeight; h++) {

            // ---- position of the 4 corners on screen ----
            rVA[currentVertex + 0].position =
                Vector2f(w * TILE,         h * TILE);
            rVA[currentVertex + 1].position =
                Vector2f((w + 1) * TILE,   h * TILE);
            rVA[currentVertex + 2].position =
                Vector2f((w + 1) * TILE,   (h + 1) * TILE);
            rVA[currentVertex + 3].position =
                Vector2f(w * TILE,         (h + 1) * TILE);

            // ---- decide which tile texture to use ----
            int texY = 0;   // top of the chosen strip on the sheet
            if (h == 0 || h == worldHeight - 1 ||
                w == 0 || w == worldWidth  - 1) {
                texY = 150;            // WALL (0,150 → 50,200)
            }
            else if (w == h) {
```

```
                texY = 50;              // GRASS (0,50 → 50,100)  diagonal
            }
            else {
                texY = 0;               // MUD-1 (0,0 → 50,50)
            }

            // ---- texture coords of the 4 corners ----
            rVA[currentVertex + 0].texCoords = Vector2f(0,    texY);
            rVA[currentVertex + 1].texCoords = Vector2f(TILE, texY);
            rVA[currentVertex + 2].texCoords = Vector2f(TILE, texY + TILE);
            rVA[currentVertex + 3].texCoords = Vector2f(0,    texY + TILE);

            currentVertex += 4;
        }
    }
    return TILE;
}
```

Why pass `VertexArray& rVA`**?** Because a VertexArray for a full arena is large (thousands of vertices). Passing by reference avoids copying it, and lets the function modify the caller's array directly.

Q8. `displayBackground` — wall border + horizontal grass mid-line

Same skeleton as Q7. Only the tile-selection logic changes: grass goes on a single horizontal row in the middle (h == worldHeight / 2), wall on the border, mud-1 everywhere else.



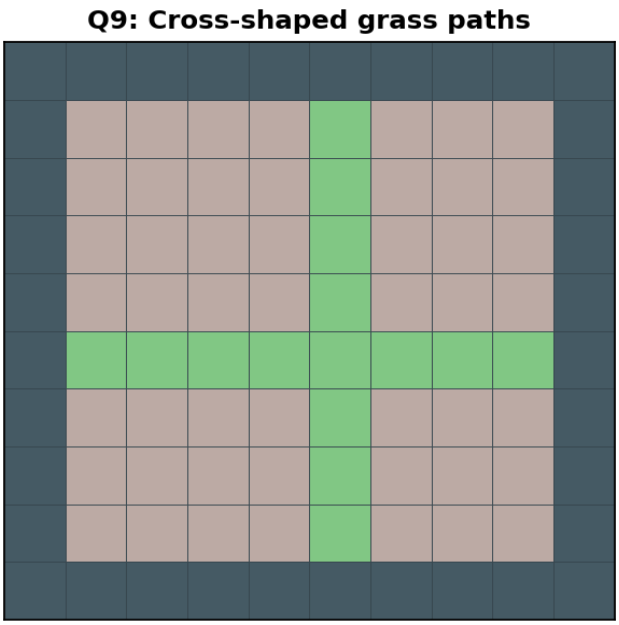
Code:

```
int displayBackground(VertexArray& rVA, IntRect arena){
    const int TILE = 50;
    int worldWidth  = arena.width  / TILE;
    int worldHeight = arena.height / TILE;

    rVA.setPrimitiveType(Quads);
    rVA.resize(worldWidth * worldHeight * 4);

    int currentVertex = 0;
    for (int w = 0; w < worldWidth;  w++) {
        for (int h = 0; h < worldHeight; h++) {

            // positions (corners of this tile)
            rVA[currentVertex + 0].position =
                Vector2f(w * TILE,       h * TILE);
            rVA[currentVertex + 1].position =
                Vector2f((w + 1) * TILE, h * TILE);
            rVA[currentVertex + 2].position =
                Vector2f((w + 1) * TILE, (h + 1) * TILE);
            rVA[currentVertex + 3].position =
                Vector2f(w * TILE,       (h + 1) * TILE);

            // tile selection — HORIZONTAL grass line
            int texY = 0;
            if (h == 0 || h == worldHeight - 1 ||
                w == 0 || w == worldWidth  - 1) {
                texY = 150;                         // WALL border
            }
            else if (h == worldHeight / 2) {
                texY = 50;                          // GRASS middle row
            }
            else {
                texY = 0;                           // MUD-1
            }

            rVA[currentVertex + 0].texCoords = Vector2f(0,    texY);
```

```
            rVA[currentVertex + 1].texCoords = Vector2f(TILE, texY);
            rVA[currentVertex + 2].texCoords = Vector2f(TILE, texY + TILE);
            rVA[currentVertex + 3].texCoords = Vector2f(0,    texY + TILE);

            currentVertex += 4;
        }
    }
    return TILE;
}
```

Q9. `displayBackground` — wall border + cross-shaped grass paths

Now grass forms a **plus / cross** shape: the middle row AND the middle column. A tile becomes grass if h == midH *or* w == midW.



Code:

```
int displayBackground(VertexArray& rVA, IntRect arena){
    const int TILE = 50;
    int worldWidth  = arena.width  / TILE;
    int worldHeight = arena.height / TILE;

    rVA.setPrimitiveType(Quads);
    rVA.resize(worldWidth * worldHeight * 4);

    int midW = worldWidth  / 2;
    int midH = worldHeight / 2;

    int currentVertex = 0;
    for (int w = 0; w < worldWidth;  w++) {
        for (int h = 0; h < worldHeight; h++) {

            // positions
            rVA[currentVertex + 0].position =
                Vector2f(w * TILE,       h * TILE);
            rVA[currentVertex + 1].position =
                Vector2f((w + 1) * TILE, h * TILE);
            rVA[currentVertex + 2].position =
                Vector2f((w + 1) * TILE, (h + 1) * TILE);
            rVA[currentVertex + 3].position =
                Vector2f(w * TILE,       (h + 1) * TILE);

            // tile selection — CROSS-shaped grass
            int texY = 0;
            if (h == 0 || h == worldHeight - 1 ||
                w == 0 || w == worldWidth  - 1) {
                texY = 150;                  // WALL border
            }
            else if (h == midH || w == midW) {
                texY = 50;                   // GRASS cross
            }
            else {
                texY = 0;                    // MUD-1
```

```
            }

            rVA[currentVertex + 0].texCoords = Vector2f(0,    texY);
            rVA[currentVertex + 1].texCoords = Vector2f(TILE, texY);
            rVA[currentVertex + 2].texCoords = Vector2f(TILE, texY + TILE);
            rVA[currentVertex + 3].texCoords = Vector2f(0,    texY + TILE);

            currentVertex += 4;
        }
    }
    return TILE;
}
```

Bonus questions from pages 11–12

The handout repeats numbering starting from Q5/Q6 on the last pages. Those are about **event polling** and **game states**, not VertexArray — but they're part of the same assignment so they're answered here too.

Q5 (p.11). Event polling for W, Return, Num0, Num1

SFML delivers user input through an event queue. Each frame we call `pollEvent` in a loop until the queue is empty. For each event of type `KeyPressed`, we check `event.key.code` against the `Keyboard::Key` enum.

Code:

```
#include <SFML/Graphics.hpp>
#include <iostream>
using namespace sf;
using namespace std;

int main(){
    RenderWindow window(VideoMode(800, 600), "Key Events");

    while (window.isOpen()) {
        Event event;
        while (window.pollEvent(event)) {

            // close window with the X button
            if (event.type == Event::Closed)
                window.close();

            // handle key presses
            if (event.type == Event::KeyPressed) {

                if (event.key.code == Keyboard::W) {
                    cout << "W key pressed (alphabet key)" << endl;
                }
                else if (event.key.code == Keyboard::Return) {
                    cout << "Return / Enter key pressed (function key)" << endl;
                }
                else if (event.key.code == Keyboard::Num0) {
                    cout << "Num0 key pressed (number key)" << endl;
                }
                else if (event.key.code == Keyboard::Num1) {
                    cout << "Num1 key pressed (number key)" << endl;
                }

                // optional: quit on Escape
                if (event.key.code == Keyboard::Escape)
                    window.close();
            }
        }

        window.clear();
        window.display();
    }
    return 0;
}
```

Polling vs real-time: `pollEvent` fires *once* per key press (good for menus, state switches). `Keyboard::isKeyPressed` returns true every frame the key is held (good for player movement).

Q6 (p.11). ON / OFF game states — player vs bloater sprite

Two states: **ON** (draw player.png) and **OFF** (draw bloater.png). The `Return` key toggles between them. Background changes to red via `window.clear(Color::Red)`.

Code:

```
#include <SFML/Graphics.hpp>
using namespace sf;

// Game state enum
enum class State { ON, OFF };

int main(){
    RenderWindow window(VideoMode(800, 600), "ON / OFF States");

    // Load textures
    Texture texturePlayer;
    texturePlayer.loadFromFile("graphics/player.png");
    Sprite spritePlayer(texturePlayer);
    spritePlayer.setPosition(300, 200);

    Texture textureBloater;
    textureBloater.loadFromFile("graphics/bloater.png");
    Sprite spriteBloater(textureBloater);
    spriteBloater.setPosition(300, 200);

    // Start in ON state
    State state = State::ON;

    while (window.isOpen()) {

        // ---- handle events ----
        Event event;
        while (window.pollEvent(event)) {
            if (event.type == Event::Closed)
                window.close();

            if (event.type == Event::KeyPressed) {
                if (event.key.code == Keyboard::Return) {
                    // toggle state
                    if (state == State::ON)
                        state = State::OFF;
                    else
                        state = State::ON;
                }
                if (event.key.code == Keyboard::Escape)
                    window.close();
            }
        }

        // ---- draw ----
        window.clear(Color::Red);     // red background

        if (state == State::ON) {
            window.draw(spritePlayer);
        }
        else {                          // State::OFF
            window.draw(spriteBloater);
        }

        window.display();
    }
    return 0;
```

```
}
```

Pattern reuse: this exact structure (an enum + a toggle on `Return` + a branch in the draw section) is how the full Zombie Arena game switches between *PLAYING*, *PAUSED*, *GAME_OVER*, and *LEVELING_UP*.

Quick reference — cheat sheet for the exam

References

```
int x = 5;
int& r = x;        // r is an alias for x — same memory
r = 10;            // x is now 10 too

// Must initialise at declaration:
int& bad;          // ERROR

// Cannot rebind:
int y = 99;
int& r2 = x;
r2 = y;            // this COPIES y into x, doesn't rebind r2
```

VertexArray of Quads — the boilerplate

```
VertexArray rVA;
rVA.setPrimitiveType(Quads);
rVA.resize(numTiles * 4);

// For each tile, write 4 vertices (TL, TR, BR, BL):
rVA[i+0].position  = Vector2f(x,          y);          // top-left
rVA[i+1].position  = Vector2f(x + TILE,   y);          // top-right
rVA[i+2].position  = Vector2f(x + TILE,   y + TILE);   // bottom-right
rVA[i+3].position  = Vector2f(x,          y + TILE);   // bottom-left

rVA[i+0].texCoords = Vector2f(0,    texY);
rVA[i+1].texCoords = Vector2f(TILE, texY);
rVA[i+2].texCoords = Vector2f(TILE, texY + TILE);
rVA[i+3].texCoords = Vector2f(0,    texY + TILE);

window.draw(rVA, &textureSheet);   // one draw call!
```

Texture-coord lookup for background_sheet.png

Tile	Top-left (u,v)	Bottom-right (u,v)	texY in code
mud-1	(0, 0)	(50, 50)	0
grass	(0, 50)	(50, 100)	50
mud-2	(0, 100)	(50, 150)	100
wall	(0, 150)	(50, 200)	150

Final tip for viva: if asked *why VertexArray over Sprite*, the answer is **one draw call**. 1000 sprites = 1000 GPU calls (slow). 1000 quads in one VertexArray = 1 GPU call (fast). The whole point of this assignment is teaching you the technique used to render the full Zombie Arena background in real time.